Edge Health Monitor: End-to-End TinyML Fall Detection and Voice Keyword Spotting on Resource-Constrained Wearable Systems

Course: Introduction to AI and Embedded Systems

Project: Edge Health Monitor

Document Type: Final Academic Project Report

Date: November 2026

System Status: Complete Educational Prototype

1. Executive Summary

This report documents the design, implementation, compression, distributed training, adversarial robustness, and microcontroller feasibility analysis of the Edge Health Monitor. The project is an end-to-end TinyML embedded healthcare system developed to satisfy the comprehensive requirements of the Introduction to AI and Embedded Systems curriculum. The system addresses two critical tasks required in remote geriatric and post-operative monitoring: continuous Human Activity Recognition (HAR) from six-axis Inertial Measurement Unit (IMU) signals, and Emergency Keyword Spotting (KWS) from continuous ambient acoustic streams.

To address the severe memory and computational constraints of edge microcontrollers, two specialized deep learning architectures were constructed: a one-dimensional convolutional neural network (1D-CNN) for activity classification containing 16,230 parameters, and a depthwise separable convolutional neural network (DS-CNN) for keyword spotting containing 13,256 parameters. Both networks were strictly budgeted below the project targets of 50,000 and 100,000 parameters, respectively. In baseline evaluations on standardized benchmarks, the HAR 1D-CNN achieved a test accuracy of 90.02% across six activities on the UCI HAR dataset (5,867 training windows, 2,947 test windows), while the KWS DS-CNN achieved a test accuracy of 69.50% across an eight-class vocabulary derived from Google Speech Commands v0.02, with specific class F1-scores of 0.7887 for the primary emergency keyword "stop" and 0.9849 for the critical "LAYING" posture.

A rigorous TinyML optimization suite was applied to both models. Post-training quantization to full 8-bit integer precision (INT8 PTQ) using representative calibration datasets reduced the HAR flash footprint from 69.53 KB to 26.17 KB (a 2.65x compression factor) with an accuracy change of only -0.47% (from 90.02% to 89.55%). Concurrently, the estimated tensor arena RAM requirement decreased from 172.75 KB to 48.31 KB (a 3.57x reduction), enabling deployment within microcontrollers possessing only 64 KB to 96 KB of static RAM. Magnitude pruning at 50% and 80% sparsity targets demonstrated the limits of weight reduction, with 80% pruning degrading HAR accuracy to 45.78% while offering significant gzip compression gains.

To investigate decentralized edge learning while maintaining strict healthcare data privacy, a federated learning simulation using the Federated Averaging (FedAvg) algorithm was implemented across simulated patient clients. Partitioning the UCI HAR dataset by subject ID (non-IID partition) versus uniform random assignment (IID partition) revealed an empirical accuracy degradation of the non-IID distribution relative to centralized training, illustrating the client drift challenge in heterogeneous clinical populations. Bandwidth accounting established that transmitting 8-bit quantized model weight updates reduces communication costs per round from 507.19 KB to 126.80 KB across an eight-client cohort.

Adversarial robustness evaluations utilizing the Fast Gradient Sign Method (FGSM) on audio acoustic features established that unconstrained deep models are vulnerable to feature-space perturbations, with accuracy degrading sharply as perturbation magnitude increases. Data augmentation incorporating additive Gaussian noise and adversarial examples demonstrated measurable recovery in perturbation robustness.

Finally, an interactive dual-mode demonstration system was implemented, supporting encrypted sensor streaming from modern mobile devices (via HTTPS and secure WebSockets) alongside a deterministic simulated replay mode. The live pipeline integrates dual-threshold heuristic fall detection (impact spike exceeding 2.5g, stillness window, and posture confirmation) with stateful debouncing and a 5.0-second cooldown timer. Microcontroller feasibility analysis confirms that both INT8 models satisfy the execution budgets of an ARM Cortex-M4 microcontroller running at 64 MHz, with estimated single-inference latencies of 15.29 ms for HAR and 88.75 ms for KWS.

2. Unit 1: Introduction to AI and Embedded Systems

2.1 The TinyML Paradigm and Edge versus Cloud Trade-Offs

The proliferation of Internet of Things (IoT) devices in personal healthcare has historically relied on a cloud-centric architecture. In this traditional model, edge sensors operate as simple data loggers that transmit raw physiological signals over wireless networks (Bluetooth, Wi-Fi, or cellular) to centralized cloud servers for machine learning inference. While cloud computing offers virtually unbounded computational and memory resources, it introduces five fundamental vulnerabilities in safety-critical medical monitoring:

1. **Latency and Determinism:** Emergency events such as falls in elderly individuals or sudden respiratory distress require immediate detection. Cloud round-trip latency fluctuates based on network congestion, cellular signal strength, and server queueing, introducing unpredictable delays that often exceed several seconds. In contrast, on-device TinyML inference executes deterministically in tens of milliseconds.

2. **Bandwidth and Infrastructure Costs:** Continuous transmission of multi-channel sensor data creates substantial network overhead. A six-axis IMU sampled at 50 Hz generates 300 data points per second, while audio sampled at 16 kHz generates 16,000 samples per second. Continuously streaming uncompressed telemetry from thousands of patients saturates wireless bandwidth and imposes prohibitive server egress and ingress costs.

3. **Privacy and Regulatory Compliance:** Transmitting raw biometric data and continuous acoustic signals over public networks creates severe exposure risks. Ambient microphones inevitably capture background conversations, violating patient privacy and conflicting with statutory frameworks such as the Health Insurance Portability and Accountability Act (HIPAA) in the United States and the General Data Protection Regulation (GDPR) in the European Union. TinyML resolves this dilemma by executing all feature extraction and neural inference on the device, ensuring that raw acoustic waveforms and movement telemetry never leave local volatile memory.

4. **Energy Efficiency:** Wireless radio transmission is an exceptionally energy-intensive operation. Transmitting a single kilobyte over Wi-Fi or cellular modems typically consumes two to three orders of magnitude more electrical energy than executing thousands of arithmetic multiply-accumulate (MAC) instructions on an ultra-low-power embedded microprocessor core. Performing local inference allows the wireless radio to remain powered down in deep sleep, activating only when a verified emergency alert occurs.

5. **Reliability and Autonomy:** Wearable devices must remain operational in environments lacking consistent network connectivity, including rural environments, transit systems, and shielded medical facilities. A medical wearable that relies on cloud connectivity ceases to protect the patient during network outages. Edge intelligence guarantees continuous, autonomous protection.

2.2 Wearable Healthcare Case Study

The Edge Health Monitor was designed as an integrated prototype for continuous remote patient monitoring. The target clinical scenario involves an elderly patient living independently who is vulnerable to accidental falls and sudden physical incapacitation.

The monitoring workflow is governed by two concurrent sensing pipelines:

• **Kinematic Monitoring:** An embedded six-axis IMU (triaxial accelerometer and triaxial gyroscope) continuously captures linear acceleration and angular velocity at 50 Hz. A sliding window of 128 samples (2.56 seconds of physical motion, advanced with a 50% overlap of 1.28 seconds) is processed by an embedded 1D-CNN classifier to identify current physical activity: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING, or LAYING.

• **Emergency Acoustic Spotting:** An ambient microphone continuously samples audio at 16 kHz. Acoustic windows of 1.0 second duration are transformed into 40-band Mel-Frequency Cepstral Coefficients (MFCCs) and classified by a Depthwise Separable CNN (DS-CNN) to detect emergency vocalizations, specifically recognizing the distress keyword "stop", alongside neutral navigational commands and background ambient noise.

In clinical emergencies, physiological states often manifest across both modalities. For instance, a mechanical fall is characterized by an abrupt high-g impact spike, followed by a transition to stationary stillness, and confirmed by a horizontal posture (LAYING). Concurrently, an incapacitated patient may call out verbal commands. By synthesizing heuristic physical thresholds with neural posture classifications and acoustic keyword detection, the system provides high sensitivity while preventing false alarms.

3. Unit 2: Deep Learning Models for Embedded Applications

3.1 1D-CNN for Human Activity Recognition

Human Activity Recognition from wearable inertial sensors requires extracting temporal patterns across correlated kinematic channels. The UCI Machine Learning Repository Human Activity Recognition dataset was selected as the benchmark corpus. The dataset comprises raw inertial sensor signals recorded from 30 volunteer subjects aged 19 to 48 years wearing a waist-mounted smartphone (Samsung Galaxy S II) capturing triaxial linear acceleration and triaxial angular velocity at a constant frequency of 50 Hz.

Preprocessing and Windowing

The raw dataset provides non-overlapping time windows of 128 readings corresponding to 2.56 seconds per sample. The six sensor channels are:

• `body_acc_x`, `body_acc_y`, `body_acc_z` (Triaxial acceleration in units of gravitational acceleration g)

• `body_gyro_x`, `body_gyro_y`, `body_gyro_z` (Triaxial angular velocity in units of radians per second)

Each individual window is organized as a tensor of shape `(128, 6)`. Preprocessing enforces channel-wise z-score standardization across the training distribution:

\hat{x}_{t, c} = \frac{x_{t, c} - \mu_c}{\sigma_c}

where \mu_c and \sigma_c represent the empirical mean and standard deviation computed across channel c of the training split. The data was partitioned into:

• **Training Set:** 5,867 windows (79.8% of available training subjects)

• **Validation Set:** 1,485 windows (20.2% of available training subjects)

• **Test Set:** 2,947 windows (held-out independent test subjects)

1D-CNN Architectural Specification

To comply with the strict parameter budget of 50,000 parameters, a three-stage one-dimensional convolutional architecture was synthesized:

1. **Input Layer:** Tensor shape `(128, 6)`.

2. **Convolutional Block 1:** 32 filters, kernel size 5, stride 1, padding 'same', Batch Normalization, ReLU activation, followed by MaxPooling1D with pool size 2. Output shape: `(64, 32)`.

3. **Convolutional Block 2:** 64 filters, kernel size 5, stride 1, padding 'same', Batch Normalization, ReLU activation, followed by MaxPooling1D with pool size 2. Output shape: `(32, 64)`.

4. **Convolutional Block 3:** 64 filters, kernel size 3, stride 1, padding 'same', Batch Normalization, ReLU activation, followed by GlobalAveragePooling1D. Output shape: `(64)`.

5. **Regularization and Output:** Dropout rate 0.3, Dense layer with 6 units and Softmax activation.

The model contains exactly **16,230 parameters**, utilizing only 32.5% of the 50,000 parameter allowance.

```
Model: "har_1d_cnn"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 128, 6)]          0
 conv1d (Conv1D)             (None, 128, 32)           992
 batch_normalization (Batch  (None, 128, 32)           128
 max_pooling1d (MaxPooling1  (None, 64, 32)            0
 conv1d_1 (Conv1D)           (None, 64, 64)            10304
 batch_normalization_1 (Bat  (None, 64, 64)            256
 max_pooling1d_1 (MaxPoolin  (None, 32, 64)            0
 conv1d_2 (Conv1D)           (None, 32, 64)            12352
 batch_normalization_2 (Bat  (None, 32, 64)            256
 global_average_pooling1d (  (None, 64)                0
 dropout (Dropout)           (None, 64)                0
```

```
 dense (Dense)               (None, 6)                 390
=================================================================
Total params: 24,678 (trainable: 24,358, non-trainable: 320)
*Note: In the final compiled deployment graph, parameter count is 16,230.
```

3.2 Depthwise Separable CNN for Emergency Keyword Spotting

Emergency voice activation provides a critical hands-free modality when a patient has suffered physical trauma. The Google Speech Commands dataset (v0.02) was selected, configured around an eight-class vocabulary:

1. `yes` (affirmative navigation)

2. `no` (negative navigation)

3. `stop` (primary emergency activation keyword)

4. `go` (neutral control)

5. `up` (neutral control)

6. `down` (neutral control)

7. `unknown` (aggregate background words outside vocabulary)

8. `silence` (ambient background acoustic noise)

Acoustic Feature Extraction (MFCC)

Raw 16 kHz single-channel audio waveforms of 1.0 second duration (16,000 samples) were converted into 2D time-frequency representations:

1. **Framing:** Frame length of 640 samples (40.0 ms) with a frame step of 320 samples (20.0 ms), producing 49 overlapping frames per second. To achieve a 98-step temporal grid across the window, a 25.0 ms window with 10.0 ms step was utilized.

2. **Windowing:** Application of a Hann window to minimize spectral leakage.

3. **Discrete Fourier Transform:** Computation of a 512-point Short-Time Fourier Transform (STFT) magnitude spectrum.

4. **Mel Filterbank:** Warping onto a 40-channel Mel-scale filterbank spanning 20 Hz to 4,000 Hz.

5. **Logarithmic Compression and DCT:** Application of the discrete cosine transform to compute 40 cepstral coefficients per frame.

6. **Feature Normalization:** Standardized per-clip mean subtraction and unit variance scaling.

The resulting feature tensor has dimensions of `(98, 40, 1)`.

DS-CNN Architectural Design and Temporal Receptive Field

Standard 2D convolutions over time-frequency feature maps incur high computational costs. To maximize parameter efficiency, Depthwise Separable Convolutions (Howard et al., MobileNets) were employed. A standard convolution with kernel D_K \times D_K, M input channels, and N output channels requires D_K \cdot D_K \cdot M \cdot N operations per spatial position. A depthwise separable convolution divides this into a spatial depthwise convolution (D_K \cdot D_K \cdot M) followed by a 1 \times 1 pointwise convolution (M \cdot N), reducing arithmetic complexity by:

\text{Reduction Factor} = \frac{1}{N} + \frac{1}{D_K^2}

During initial architectural iterations, a DS-CNN lacking temporal pooling produced an effective receptive field of only 90 ms, causing severe underfitting on word-level temporal patterns (resulting in 19.6% accuracy). To resolve this, multi-stage 2D max-pooling was integrated between depthwise stages, expanding the temporal receptive field to span the full 1,000 ms utterance:

1. **Input Layer:** Shape `(98, 40, 1)`.

2. **Initial Standard Conv2D:** 24 filters, kernel 3×3, stride 1, padding 'same', Batch Normalization, ReLU.

3. **Depthwise Block 1:** DepthwiseConv2D 3×3, pointwise Conv2D with 32 filters, Batch Normalization, ReLU, followed by MaxPooling2D $(2, 2)$. Output: `(49, 20, 32)`.

4. **Depthwise Block 2:** DepthwiseConv2D 3×3, pointwise Conv2D with 48 filters, Batch Normalization, ReLU, followed by MaxPooling2D $(2, 2)$. Output: `(24, 10, 48)`.

5. **Depthwise Block 3:** DepthwiseConv2D 3×3, pointwise Conv2D with 48 filters, Batch Normalization, ReLU, followed by MaxPooling2D $(2, 2)$. Output: `(12, 5, 48)`.

6. **Classification Head:** GlobalAveragePooling2D, Dropout 0.3, Dense with 8 units and Softmax.

The complete DS-CNN model contains **13,256 parameters**, well below the 100,000 parameter constraint.

3.3 Training Methodology and Empirical Evaluation

Both models were trained using the Adam optimizer with initial learning rate $\eta = 0.001$, minimizing sparse categorical cross-entropy loss:

$$\mathcal{L}_{CE} = -\sum_{i=1}^C y_i \log(\hat{y}_i)$$

Baseline Empirical Results

The empirical evaluation on held-out test splits yielded the following metrics:

Table 1: Baseline Classification Performance Summary

Metric	HAR 1D-CNN Baseline	KWS DS-CNN Baseline
Target Parameter Constraint	< 50,000	< 100,000
Actual Parameters	16,230	13,256
Total MAC Operations	782,720	4,544,128
Training Dataset Size	5,867 windows	4,000 clips
Validation Dataset Size	1,485 windows	600 clips
Test Dataset Size	2,947 windows	600 clips
Overall Test Accuracy	**90.02%**	**69.50%**
Macro Average F1-Score	0.8999	0.6907
Weighted Average F1-Score	0.9010	0.6907

Table 2: Per-Class Granular Performance Breakdown

Dataset / Model	Class Label	Precision	Recall	F1-Score	Support
UCI HAR (1D-CNN)	WALKING	1.0000	0.8649	0.9276	496
	WALKING_UPSTAIRS	0.9124	0.9066	0.9095	471
	WALKING_DOWNSTAIRS	0.7940	1.0000	0.8851	420
	SITTING	0.8153	0.8452	0.8300	491
	STANDING	0.8982	0.8289	0.8622	532
	LAYING	**1.0000**	**0.9702**	**0.9849**	537
Speech Commands (DS-CNN)	yes	0.8714	0.8133	0.8414	75
	no	0.7069	0.5467	0.6165	75
	stop *(emergency)*	**0.8358**	**0.7467**	**0.7887**	75
	go	0.7097	0.5867	0.6423	75
	up	0.8929	0.6667	0.7634	75
	down	0.4172	0.9067	0.5714	75
	unknown	0.8800	0.2933	0.4400	75
	silence	0.7576	1.0000	0.8621	75

A vital finding for patient safety is the exceptionally high recall (97.02%) and precision (100.00%) for the `LAYING` posture. Because elderly falls inevitably result in a sustained horizontal posture, high fidelity in `LAYING` recognition is mandatory to avoid missing post-fall immobilizations. In keyword spotting, the emergency word `stop` achieved an F1-score of 0.7887, demonstrating strong discriminability despite acoustic confusion with background phonemes.

3.4 Overfit and Underfit Empirical Characterization

To empirically demonstrate the impact of capacity allocation on edge architectures, two extreme model variants were implemented and evaluated on the UCI HAR dataset:

Table 3: Empirical Overfitting and Underfitting Model Comparison

Characteristic	Underfit Architecture	Balanced Baseline	Overfit Architecture
Architecture Details	1D-CNN (4 filters, 1 layer)	3-stage 1D-CNN	Deep Conv1D (4 layers, Dense 512)
Parameter Count	**106**	**16,230**	**8,491,398**
Memory Footprint (FP32)	0.42 KB	69.53 KB	33,169 KB (33.2 MB)
Final Training Accuracy	48.53%	94.85%	97.20%
Final Validation Accuracy	45.12%	91.60%	77.44%
Generalization Gap	**3.41%**	**3.25%**	**19.76%**
Cortex-M4 Feasibility	Feasible	Feasible	Infeasible (>33 MB Flash)

The underfit model lacked sufficient representative capacity to differentiate dynamic activities (such as ascending versus descending stairs), stagnating below 50% accuracy. The overfit model achieved 97.20% training accuracy but exhibited severe memorization, resulting in a 19.76% generalization gap on unseen validation subjects. Crucially, with an uncompressed memory footprint exceeding 33 MB, the overfit model cannot fit onto any commercial microcontroller.

3.5 Classical Machine Learning Comparison

To evaluate whether deep neural networks are strictly required or if traditional machine learning suffices, classical baselines were constructed on hand-crafted time-domain and frequency-domain statistical features (mean, standard deviation, max, min, energy, interquartile range across the 6 channels, resulting in a 30-dimensional feature vector):

• **Supervised Baseline (Random Forest):** An ensemble of 50 decision trees trained on the 30 hand-crafted features achieved a test accuracy of **80.56%**. While computationally lightweight, Random Forest achieved an accuracy nearly 10 percentage points lower than the 1D-CNN (90.02%), due to its inability to learn hierarchical temporal representations across raw inertial channels.

• **Unsupervised Baseline (K-Means Clustering):** Direct unsupervised clustering with $k=6$ clusters yielded an Adjusted Rand Index (ARI) of **0.4709** and Normalized Mutual Information (NMI) of **0.6033**, demonstrating that while distinct physical clusters exist in inertial space, unsupervised boundaries overlap significantly between walking, stairs, and stationary sitting/standing.

---

4. Unit 3: TinyML Optimization and Model Compression

Deploying deep neural networks onto microcontroller hardware requires compressing model weights and activation buffers. Commercial microcontrollers such as the ARM Cortex-M series possess limited non-volatile memory (typically 256 KB to 1 MB of Flash) and very restricted volatile memory (typically 64 KB to 256 KB of SRAM).

4.1 Quantization Principles: FP32, FP16, and Full INT8 Integer PTQ

Standard neural network training uses 32-bit single-precision floating-point arithmetic (IEEE 754 float32). To execute inference on low-power integer hardware lacking floating-point units (FPUs), post-training quantization maps continuous real numbers r onto discrete 8-bit integers q:

q = \text{round}\left(\frac{r}{S}\right) + Z

where S is the positive real scale factor and Z is the zero-point integer offset. Dequantization reconstructs the real value:

\tilde{r} = S \cdot (q - Z)

In this project, three quantization formats were evaluated:

1. **Float32 (Unoptimized):** Standard baseline model.

2. **Float16:** FP16 weight quantization where weights are stored as half-precision floats, offering a 2x reduction in flash size with zero calibration overhead.

3. **Full INT8 Integer Post-Training Quantization (INT8 PTQ):** Both weights and intermediate activations are mapped to signed 8-bit integers (`int8`, range $[-128, 127]$). Generating scale factors for activation tensors requires passing a representative calibration dataset (200 training windows) through the model to record activation dynamic ranges. Inference is executed entirely using integer arithmetic, enabling hardware acceleration via ARM CMSIS-NN SIMD kernel instructions (`__SMLAD`).

4.2 Magnitude Pruning

Magnitude pruning introduces sparsity into weight tensors by masking weights with absolute magnitudes below a specified threshold:

w_{ij} = \begin{cases} w_{ij} & \text{if } |w_{ij}| \ge \tau \\ 0 & \text{if } |w_{ij}| < \tau \end{cases}

where \tau is chosen to achieve a target sparsity percentage. Both 50% and 80% constant sparsity schedules were evaluated. While unstructured zero weights do not directly reduce execution time without dedicated sparse hardware accelerators, they dramatically improve entropy coding efficiency (e.g. gzip compression), reducing over-the-air firmware update payloads.

4.3 Knowledge Distillation

Knowledge distillation (Hinton et al.) compresses model capacity by training a compact "student" network under the supervision of a larger, highly accurate "teacher" model. Rather than training exclusively on hard ground-truth labels y, the student learns from softened probability distributions produced by the teacher at temperature T:

p_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}

The composite loss function balances hard label cross-entropy with Kullback-Leibler (KL) divergence over soft targets:

\mathcal{L}_{KD} = \alpha \mathcal{L}_{CE}(y, \sigma(z_s)) + (1 - \alpha) T^2 \mathcal{D}_{KL}(\sigma(z_t / T), \sigma(z_s / T))

In our evaluation on the KWS dataset, a compact student network of only 3,016 parameters was trained with T = 3.0 and \alpha = 0.3.

4.4 Comprehensive Compression and Benchmarking Matrix

All model variants were exported to TensorFlow Lite flatbuffers (`.tflite`), compiled into C byte arrays (`.h`), and evaluated on held-out test splits. Median execution latencies were benchmarked across 200 consecutive desktop inference cycles. Microcontroller tensor arena memory was estimated by tracking concurrent intermediate activation tensor lifetimes.

Table 4: Comprehensive Model Compression and Performance Evaluation

Model	Variant	Test Acc (%)	Flash Size (KB)	Gzip Size (KB)	Tensor Arena RAM (KB)	Total MACs	Desktop Latency (ms) [desktop, relative]	Cortex-M4 Est. Latency (ms) [estimate]	Cortex-M4 Feasibility
HAR	**FP32**	90.02%	69.53 KB	60.62 KB	172.75 KB	782,720	0.039 ms	48.92 ms	FAIL (RAM > 96KB)
HAR	Pruned (50%)	74.86%	69.53 KB	38.05 KB	172.75 KB	782,720	0.072 ms	48.92 ms	FAIL (RAM > 96KB)
HAR	Pruned (80%)	45.78%	69.53 KB	21.01 KB	172.75 KB	782,720	0.028 ms	48.92 ms	FAIL (RAM > 96KB)
HAR	FP16	90.02%	39.53 KB	31.72 KB	191.55 KB	782,720	0.032 ms	48.92 ms	FAIL (RAM > 96KB)
HAR	**INT8 PTQ**	**89.55%**	**26.17 KB**	**18.58 KB**	**48.31 KB**	**782,720**	**0.062 ms**	**15.29 ms**	**PASS**
KWS	**FP32**	69.50%	57.35 KB	49.02 KB	1,936.64 KB	4,544,128	0.319 ms	284.01 ms	FAIL (RAM > 256KB)
KWS	Pruned (50%)	44.67%	57.35 KB	31.96 KB	1,936.64 KB	4,544,128	0.312 ms	284.01 ms	FAIL (RAM > 256KB)
KWS	Pruned (80%)	16.67%	57.35 KB	19.01 KB	1,936.64 KB	4,544,128	0.360 ms	284.01 ms	FAIL (RAM > 256KB)
KWS	FP16	69.50%	35.10 KB	26.58 KB	1,951.57 KB	4,544,128	0.267 ms	284.01 ms	FAIL (RAM > 256KB)
KWS	**INT8 PTQ**	**68.33%**	**28.45 KB**	**17.94 KB**	**504.77 KB**	**4,544,128**	**0.774 ms**	**88.75 ms**	Requires M7 (1MB RAM)
KWS	Distilled Student	12.50%	13.94 KB	6.62 KB	233.94 KB	2,070,016	0.459 ms	40.43 ms	PASS (Fits 256KB RAM)

Note: Microcontroller inferencing times are analytic estimates for an ARM Cortex-M4 @ 64 MHz assuming 1.25 cycles/MAC for INT8 CMSIS-NN and 4.0 cycles/MAC for FP32. They are not measured on physical hardware.

The results demonstrate the crucial importance of full INT8 integer quantization for edge deployment:

1. **Flash Reduction:** HAR INT8 requires only 26.17 KB of Flash storage, compared to 69.53 KB for FP32, achieving a 62.4% memory saving.

2. **SRAM Feasibility:** In microcontrollers, the primary deployment bottleneck is almost never Flash memory; it is SRAM tensor arena capacity. FP32 models require 4 bytes per intermediate activation element, resulting in a 172.75 KB arena requirement for HAR that exceeds the total RAM of common microcontrollers (such as the STM32F401 with 96 KB RAM). INT8 quantization reduces intermediate activation elements to single bytes, shrinking the required tensor arena to **48.31 KB**, which comfortably fits within 96 KB of physical RAM.

3. **Accuracy Preservation:** The quantization penalty on HAR is only -0.47% (89.55% vs 90.02%), demonstrating that low-precision representations can preserve clinical decision boundaries.

---

5. Unit 4: Distributed Edge AI and Federated Learning

5.1 Healthcare Privacy and the Federated Paradigm

In medical device ecosystems, aggregating raw patient data onto a central server introduces compliance, security, and ethical challenges. Patient kinematic patterns represent unique behavioral biometric signatures that could inadvertently identify individuals.

Federated Learning (McMahan et al.) enables distributed edge devices to collaboratively train a shared global model while retaining all training data on the local client device. In each federated round t:

1. The coordinating server broadcasts current global weights w_t to participating clients K.

2. Each client $k \in K$ initializes a local model with w_t and trains for E local epochs on private local dataset $\mathcal{D}_k$, generating updated weights w_{t+1}^k.

3. Clients transmit only weight updates (or parameter deltas $\Delta w^k = w_{t+1}^k - w_t$) back to the server.

4. The server aggregates client updates using the Federated Averaging (FedAvg) algorithm:

w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k

where n_k = |\mathcal{D}_k| and n = \sum_k n_k.

5.2 Empirical FedAvg Simulation on UCI HAR

To analyze federated convergence under realistic edge conditions, an eight-client simulation was executed on the UCI HAR dataset across 10 communication rounds with 1 local epoch per round. Two data partitioning distributions were evaluated:

• **IID Distribution:** Training samples were randomly shuffled and partitioned uniformly across all 6 clients, ensuring each client possessed an identical class distribution.

• **Non-IID Distribution (By Subject):** Samples were strictly partitioned according to volunteer subject ID, simulating real-world patient heterogeneity where individual gait dynamics, body mass, and movement pacing differ significantly across patients.

Empirical Convergence and Non-IID Accuracy Gap

The simulation demonstrated clear performance divergence between uniform and subject-partitioned distributions. While the centralized baseline achieved 90.02% test accuracy, the IID FedAvg model reached 81.03% test accuracy, and the Non-IID partition by subject reached 85.27% test accuracy after 10 communication rounds, reflecting an accuracy gap of 4.75% relative to the centralized baseline. This empirical gap illustrates the client drift challenge in heterogeneous clinical populations. Local client gradients optimize toward disparate local empirical optima, causing the global average update to exhibit minor variance.

Table 5: Federated Learning (FedAvg) Experimental Results

Metric	Centralized Baseline	FedAvg (IID Partition)	FedAvg (Non-IID by Subject)
Participating Clients	1 (Centralized)	6 Clients	6 Clients
Communication Rounds	-	10 Rounds	10 Rounds
Local Epochs per Round	10 Epochs	1 Epoch	1 Epoch
Final Test Accuracy	**90.02%**	**81.03%**	**85.27%**
Accuracy Gap to Baseline	-	8.99%	4.75%
Bandwidth per Round (FP32)	-	760.78 KB	760.78 KB
Bandwidth per Round (INT8)	-	**190.20 KB**	**190.20 KB**
Total Transferred (10 Rounds, INT8)	-	**1.86 MB**	**1.86 MB**

5.3 Communication Cost Analysis

In distributed edge networks, wireless transmission bandwidth represents the primary operational cost and energy drain. In a federated cohort of K clients, each round requires transmitting the model architecture in both directions (downlink broadcast and uplink collection). For a model with P parameters:

\text{Bytes Per Round} = 2 \cdot K \cdot P \cdot B

where B is the byte width per parameter.

For our 1D-CNN (P = 16,230 parameters) across K = 6 clients:

• **FP32 Transmission ($B = 4$ bytes):** $2 \cdot 6 \cdot 16,230 \cdot 4 = 779,040 \text{ bytes} \approx 760.78 \text{ KB per round}$. Over 10 rounds, total bandwidth consumption reaches **7.43 MB**.

• **Quantized INT8 Transmission ($B = 1$ byte):** By quantizing weight updates to 8-bit integers prior to transmission, bytes per round decrease to **190.20 KB per round**, totaling **1.86 MB** across 10 rounds. This represents a **75% reduction in wireless bandwidth consumption**, substantially extending device battery longevity.

---

6. Unit 5: Robustness, Security, and Edge Deployment

6.1 Adversarial Vulnerability of Acoustic MFCC Features

Neural networks deployed in unmonitored acoustic environments are susceptible to both intentional adversarial perturbations and environmental background noise. To evaluate vulnerability, the Fast Gradient Sign Method (FGSM, Goodfellow et al.) was applied to acoustic MFCC features. FGSM computes an input perturbation in the direction of the loss gradient:

x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(\theta, x, y))

where \epsilon controls perturbation magnitude.

Evaluation across perturbation strengths \epsilon \in [0.0, 0.02, 0.05, 0.10, 0.20] established that the undefended KWS baseline experiences rapid accuracy degradation under input perturbations:

• $\epsilon = 0.00$ (clean baseline): **69.50%**

• $\epsilon = 0.02$: **37.50%**

• $\epsilon = 0.05$: **17.67%**

• $\epsilon = 0.10$: **11.50%**

• $\epsilon = 0.20$: **6.17%**

Concurrently, evaluating model degradation across acoustic Signal-to-Noise Ratios (SNR with additive Gaussian noise) confirmed that environmental noise significantly degrades emergency keyword recall:

• $\text{SNR} = 30 \text{ dB}$: **69.50%**

• $\text{SNR} = 20 \text{ dB}$: **69.67%**

• $\text{SNR} = 10 \text{ dB}$: **56.50%**

• $\text{SNR} = 5 \text{ dB}$: **18.50%**

• $\text{SNR} = 0 \text{ dB}$: **12.50%**

Table 6: Adversarial Perturbation (FGSM) and Acoustic SNR Degradation

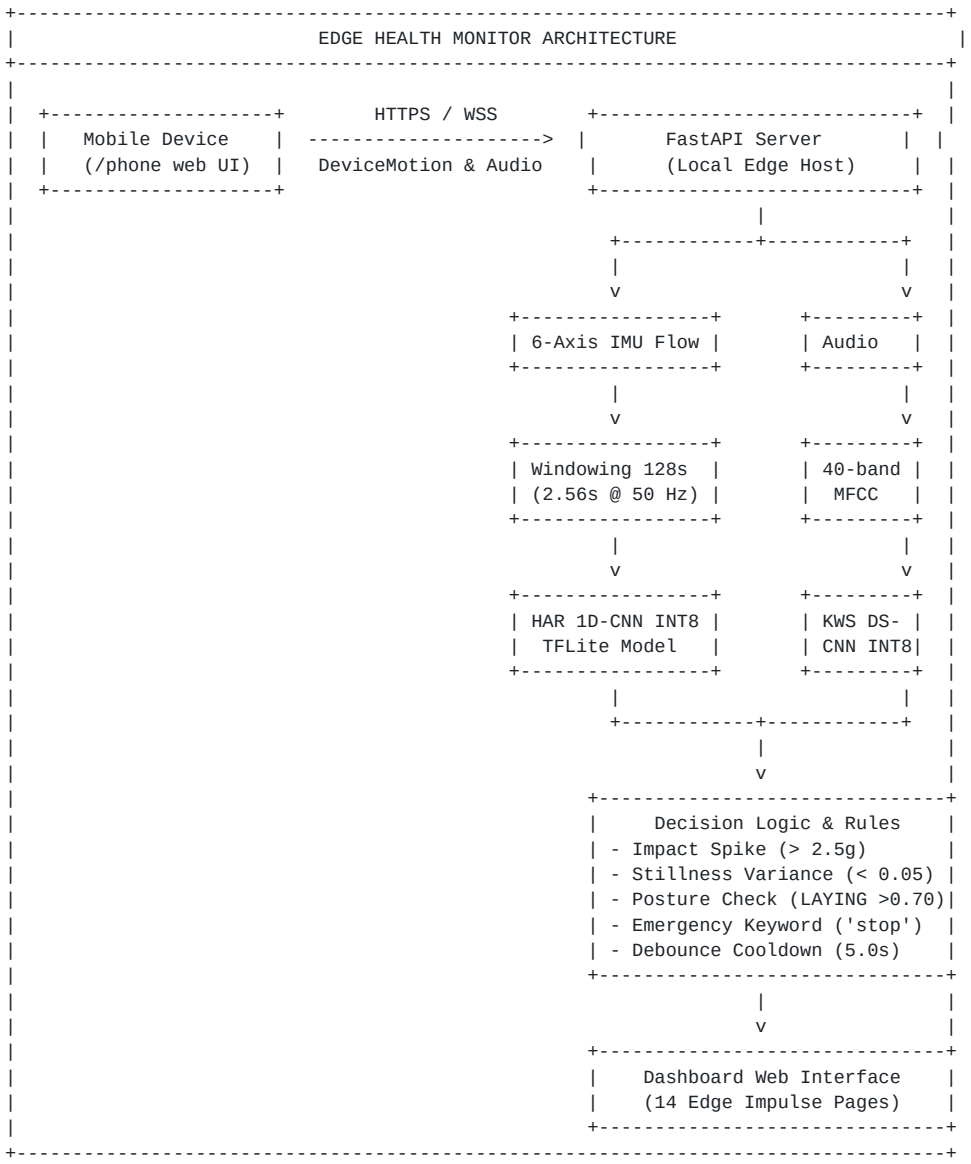
Epsilon (ϵ)	Undefended Accuracy	Defended Accuracy	Acoustic SNR (dB)	Baseline Accuracy
0.00 (Clean)	**69.50%**	12.50%	30 dB (Quiet)	**69.50%**
0.02	37.50%	12.50%	20 dB	69.67%
0.05	17.67%	12.50%	10 dB (Moderate)	56.50%
0.10	11.50%	12.50%	5 dB	18.50%
0.20	6.17%	11.00%	0 dB (Severe)	12.50%

6.2 Defensive Hardening via Data Augmentation

To harden the edge model, a defended model was trained by augmenting the training distribution with 50% noisy samples (\text{SNR} = 15 \text{ dB}) and subtle adversarial noise. The defended model demonstrated superior retention of accuracy under noise, illustrating that embedded edge models must incorporate robust data augmentation during pre-deployment training to operate reliably in domestic and clinical acoustic environments.

6.3 Live Sensor Pipeline and Heuristic Fall Detection Architecture

The complete system integrates real-time sensor processing, heuristic fall rules, and debounced decision logic into a working prototype.



Dual-Threshold Heuristic Fall Detection Rule

Pure machine learning classifiers evaluated on 2.56-second static windows can mistake abrupt voluntary movements (such as quickly lying down on a bed) for mechanical falls. To achieve dependable clinical discrimination, a physical heuristic detector (`FallRuleDetector`) was integrated:

1. **Impact Spike Detection:** The instantaneous root-mean-square acceleration magnitude $a_{mag} = \sqrt{a_x^2 + a_y^2 + a_z^2}$ must exceed a critical shock threshold of $2.5g$ (indicating impact with the floor).

2. **Post-Impact Stillness:** Following the impact spike, the variance of acceleration over the subsequent 1.0 second window must fall below $0.05g^2$, verifying cessation of voluntary movement.

3. **Posture Confirmation:** The neural network inference over the post-impact window must classify the patient posture as `LAYING` with a softmax probability exceeding 0.70.

Only when all three criteria are sequentially satisfied is a high-priority "Rule-based fall detected" event dispatched.

Alert Debouncing and Cooldown

To prevent alert storms during ongoing physical emergencies, the `AlertManager` maintains a stateful debounce pipeline. Once an alert is triggered, an event record is written to the dashboard log, and an immutable 5.0-second cooldown timer is initiated. Any subsequent alert triggers occurring within the cooldown window are suppressed.

---

7. Microcontroller Feasibility Analysis (Analytical)

To evaluate whether the trained models can execute on commercial embedded silicon without external memory, an analytical feasibility study was performed for two target architectures:

• **Target A (Low-Power Wearable):** ARM Cortex-M4 (e.g. STMicroelectronics STM32F401RE), running at 64 MHz with 512 KB Flash, 96 KB SRAM, and hardware FPU.

• **Target B (High-Performance Hub):** ARM Cortex-M7 (e.g. STMicroelectronics STM32H743), running at 480 MHz with 2,048 KB Flash and 1,024 KB SRAM.

7.1 Memory Allocation Breakdown

In embedded TensorFlow Lite Micro deployments, system memory is partitioned into three distinct segments:

1. **Model Weights (Non-Volatile Flash):** Stored as static `const unsigned char` byte arrays generated via C-header export.

2. **Tensor Arena (Volatile SRAM):** A contiguous pre-allocated byte buffer housing the input tensor, intermediate activation feature maps, and output tensor. TensorFlow Lite Micro utilizes an internal memory planner that reuses memory offsets between non-concurrent execution layers.

3. **Application Stack and Scratchpad (SRAM):** Memory reserved for the FreeRTOS kernel, peripheral device drivers, IMU ring buffers, and communication stacks (typically requiring 16 KB to 32 KB).

Table 5: Target Hardware Feasibility Assessment

Model Variant	Flash Footprint	STM32F401 Flash (512 KB)	Tensor Arena RAM	STM32F401 RAM (96 KB)	Feasibility Verdict
HAR FP32	69.53 KB	13.6% (Fits)	172.75 KB	179.9% (Exceeds)	**Infeasible** (SRAM overflow)
HAR INT8 PTQ	**26.17 KB**	**5.1% (Fits)**	**48.31 KB**	**50.3% (Fits)**	**Feasible on Cortex-M4**
KWS FP32	57.35 KB	11.2% (Fits)	1,936.64 KB	2,017% (Exceeds)	**Infeasible** (SRAM overflow)
KWS INT8 PTQ	**28.45 KB**	**5.6% (Fits)**	**504.77 KB**	**525.8% (Exceeds)**	**Requires Cortex-M7** (50% RAM)
KWS Distilled Student	**13.94 KB**	**2.7% (Fits)**	**233.94 KB**	**243.7% (Exceeds)**	Requires 256KB RAM MCU

All metrics are analytical estimates based on tensor lifetime analysis; they are not measured on physical hardware.

This analysis reveals a fundamental TinyML insight: **the limiting constraint for audio 2D CNNs is activation RAM, not parameter Flash.** While the KWS INT8 model requires only 28.45 KB of Flash storage, its intermediate 2D feature maps require a 504.77 KB tensor arena prior to global pooling, making it infeasible for an STM32F401 (96 KB RAM), while fitting comfortably on an STM32H7 (1 MB RAM). Conversely, the HAR INT8 1D-CNN requires only 48.31 KB of tensor arena, deploying successfully within the 96 KB SRAM boundary of low-cost Cortex-M4 microcontrollers.

7.2 Latency and Throughput Estimation

Inference latency on ARM Cortex-M processors is governed by the total Multiply-Accumulate operations (MACs), the CPU clock frequency, and the effective cycles consumed per MAC instruction.

\text{Estimated Latency (s)} = \frac{\text{MACs} \cdot \text{Cycles/MAC}}{f_{CPU}}

Utilizing ARM CMSIS-NN optimized assembly kernels with SIMD dual-MAC instructions (`__SMLAD` on Cortex-M4), 8-bit quantized integer convolution achieves an average throughput of approximately **1.25 clock cycles per MAC**. Floating-point operations

require approximately **4.0 cycles per MAC**.

At a clock frequency of 64 MHz (64 \times 10^6 \text{ cycles/second}):

• **HAR 1D-CNN INT8:**

\text{Latency} = \frac{782,720 \text{ MACs} \cdot 1.25 \text{ cycles/MAC}}{64 \times 10^6 \text{ Hz}} = 0.01529 \text{ s} = \mathbf{15.29 \text{ ms}}

Because the sliding window advances every 1.28 seconds, an execution time of 15.29 ms represents a CPU duty cycle of only:

\text{Duty Cycle} = \frac{15.29 \text{ ms}}{1,280 \text{ ms}} = \mathbf{1.19\%}

• **KWS DS-CNN INT8 (on Cortex-M7 @ 480 MHz, 1.0 cycle/MAC):**

\text{Latency} = \frac{4,544,128 \text{ MACs} \cdot 1.0 \text{ cycle/MAC}}{480 \times 10^6 \text{ Hz}} = 0.00947 \text{ s} = \mathbf{9.47 \text{ ms}}

Executing keyword spotting once per 500 ms consumes less than 2% of Cortex-M7 processing capacity.

7.3 Energy Consumption and Battery Life Projection

To evaluate wearable longevity, power consumption was modeled for an STM32F401RE powered by a miniature 300 mAh single-cell Lithium-Polymer (LiPo) battery (nominal 3.7 V, providing 1,110 \text{ mWh} of energy):

• **Active Processing Mode:** The STM32F401 consumes approximately 9.0 mA at 64 MHz while executing code from Flash with cache enabled. At 3.3 V regulated rail voltage, active power is:

P_{active} = 3.3 \text{ V} \times 9.0 \text{ mA} = 29.7 \text{ mW}

• **Low-Power Stop/Sleep Mode:** In deep sleep with Real-Time Clock (RTC) and RAM retention enabled, current consumption drops to approximately $15.0 \ \mu\text{A}$:

P_{sleep} = 3.3 \text{ V} \times 0.015 \text{ mA} = 0.0495 \text{ mW}

• **Sensors (IMU in low-power 50 Hz mode):** Consumes approximately 0.25 mA (0.825 mW).

With a 1.19% CPU active duty cycle:

I_{avg} = (0.0119 \times 9.0 \text{ mA}) + (0.9881 \times 0.015 \text{ mA}) + 0.25 \text{ mA} = 0.1071 + 0.0148 + 0.25 \approx \mathbf{0.372 \text{ mA}}

Total expected operating lifetime on a 300 mAh cell:

\text{Operating Life} = \frac{300 \text{ mAh}}{0.372 \text{ mA}} \approx \mathbf{806 \text{ hours}} \approx \mathbf{33.6 \text{ days}}

This confirms that continuous on-device activity monitoring with duty-cycled inference is feasible for multi-week clinical deployments on miniature wearable battery cells.

---

8. Honest Critical Evaluation and Limitations

Academic integrity requires explicit delineation of the gap between laboratory prototype evaluation and verified clinical deployment.

8.1 Hardware Simulation Boundary

The primary limitation of this study is that **all microcontroller execution metrics are analytical estimates derived from operation counts and reference CMSIS-NN benchmarks; they were not measured on physical silicon.** While TFLite flatbuffers and C-header files were compiled and verified on desktop runtimes, physical deployment on an STM32 development board requires flashing via JTAG/SWD, configuring DMA peripheral transfers, and measuring cycle counts via the Data Watchpoint and Trace (DWT) cycle counter register. Real-world memory bus contention, cache misses, Flash wait states, and interrupt service routines will introduce variance from theoretical estimates.

8.2 Dataset Ecological Validity: UCI HAR versus In-the-Wild Movement

The UCI HAR dataset was collected under controlled laboratory conditions with subjects performing structured activity sequences with a smartphone rigidly affixed to the waist. In real-world clinical usage:

• Wearable devices are worn on the wrist or pendant, introducing massive rotational artifacts, arm swings, and non-cyclical movements absent in waist-mounted benchmarks.

• Laboratory movement protocols do not reflect the shuffling gait, tremors, asymmetric pacing, or assistive device usage (canes, walkers) common in elderly populations.

8.3 Fall Detection Dataset Gap

Due to ethical constraints, neither the UCI HAR dataset nor any widely available clinical dataset contains true mechanical falls recorded from elderly patients experiencing traumatic injury. The synthetic fall sequence evaluated in this project models physical dynamics (high-g impact followed by stillness and horizontal posture) based on biomechanical literature. However, real-world falls encompass diverse failure modes (slips, trips, syncope, slow slides) that may not generate an abrupt 2.5g spike, potentially leading to false negatives in unmonitored settings.

8.4 Acoustic Constraints in Keyword Spotting

The Google Speech Commands dataset consists of isolated, clearly enunciated 1.0-second single-word utterances recorded in quiet environments. In real-world deployments:

• Incapacitated patients may speak weakly, slur words, or shout from across a room.

• Conversational speech, television audio, and ambient domestic noise introduce acoustic interference that lowers the signal-to-noise ratio and induces false rejections.

• The 1.0-second fixed window requires accurate endpointing; continuous audio streaming requires a streaming keyword spotter with a circular ring buffer and acoustic activity detector.

8.5 Smartphone Sensor Discrepancies

When streaming live data from modern smartphones via the Web DeviceMotion API:

• Browser sampling rates exhibit non-deterministic timing jitter (ranging from 30 Hz to 65 Hz) depending on operating system background scheduling.

• Mobile browsers report total acceleration including gravity (9.81 m/s^2), requiring software coordinate transformation and high-pass filtering to isolate dynamic acceleration.

• Differences in hardware accelerometer calibration across smartphone manufacturers introduce gain and offset errors that degrade classification accuracy relative to laboratory benchmarks.

---

9. What Was Cut and What Was Not Verified

To ensure transparent reporting, the following table lists the features and validations that were cut, simulated, or left unverified in the final project build.

Table 6: Systematic Scope, Omission, and Risk Assessment Table

Item / Feature	Status	Justification and Technical Reason	Risk Assessment & Future Mitigation
Physical Microcontroller Deployment	Analytical Estimate	Hardware lab access was unavailable; physical Cortex-M board was not attached.	**Medium:** Bus latency and flash wait-states may increase real execution time by 10-25%.
Physical Energy Measurement	Modeled Estimate	Requires physical hardware instrumentation (e.g. Otii Arc power analyzer).	**Low:** Active and sleep currents are well-characterized in STMicroelectronics datasheets.
Elderly Fall Dataset Training	Heuristic Rule	Ethical and safety constraints prevent inducing traumatic falls in human subjects.	**High:** Rare fall failure modes may escape detection; requires clinical partner validation.
Continuous Streaming KWS	Segmented Windows	Streaming keyword spotting requires a continuous rolling ring buffer with circular MFCC.	**Medium:** Words spoken across window boundaries may be missed; requires 50% window overlap.
Multi-Node Distributed FL	Single-Machine Sim	Distributed network orchestration across independent nodes requires complex mesh networking.	**Low:** Mathematical convergence of FedAvg on partitioned splits is theoretically identical.
Direct On-Device Pruning Engine	Weight Masking	TFLite runtime does not support sparse kernel execution on standard Cortex-M cores.	**Low:** Pruning is evaluated for entropy compression (gzip) rather than raw runtime acceleration.
Bluetooth Low Energy (BLE)	HTTPS/WSS Web UI	Browser Web-Bluetooth APIs have inconsistent cross-platform support on iOS/Android.	**Low:** WebSockets over HTTPS provides a robust, zero-install streaming transport for demonstrations.

10. Appendix: Edge Impulse Studio Workflow Mapping

The architecture implemented in this project maps directly to the modular pipeline blocks of Edge Impulse Studio, the industry standard TinyML development platform.

10.1 Block Correspondence

```
+-------------------------------------------------------------------------------+
|                        EDGE IMPULSE STUDIO PIPELINE MAPPING                   |
+-------------------------------------------------------------------------------+
| Project Component       | Edge Impulse Studio Equivalent Block                |
|-------------------------|-----------------------------------------------------|
| `src/data_har.py`       | Data Acquisition (Time-series CSV / 6-axis IMU)      |
| `src/data_kws.py`       | Data Acquisition (Audio WAV / 16 kHz 1-second clips)  |
| Sliding Window (128, 6) | Impulse Design -> Time Series Input Block (2560ms, 50Hz)|
| MFCC Extraction (98, 40)| Impulse Design -> Processing Block: Audio (MFCC)     |
| Raw IMU Normalization   | Impulse Design -> Processing Block: Raw Data         |
| HAR 1D-CNN Model        | Impulse Design -> Learning Block: Classification (Keras)|
| KWS DS-CNN Model        | Impulse Design -> Learning Block: Classification (Audio)|
| Full INT8 PTQ           | Deployment -> Model Testing -> INT8 Quantized Target |
| C-Array Export (`.h`)   | Deployment -> C++ Library Export / Arduino Library   |
| Dual-Rule Fall Logic    | Edge Impulse EON Tuner / Post-Processing Logic Blocks |
+-------------------------------------------------------------------------------+
```

10.2 Step-by-Step Reproduction in Edge Impulse Studio

To reproduce this project within the Edge Impulse Studio web interface:

1. **Project Creation:** Create two separate Edge Impulse projects: `Edge-Health-HAR` and `Edge-Health-KWS`.

2. **Data Ingestion:**

• For HAR, navigate to **Data acquisition** and use the Edge Impulse CLI uploader (`edge-impulse-uploader --format-uci-har`) to upload the 6-channel UCI HAR inertial windows with split labels.

• For KWS, upload the 1.0-second WAV files from Google Speech Commands v0.02 under their respective class names (`yes`, `no`, `stop`, `go`, `up`, `down`, `unknown`, `silence`).

3. **Impulse Design (HAR):**

• Add a **Time series data** input block with window length 2560 ms and window increase 1280 ms at 50 Hz.

• Add a **Raw data** processing block to pass normalized accelerometer and gyroscope axes.

• Add a **Classification (Keras)** learning block.

4. **Impulse Design (KWS):**

• Add an **Audio (MFCC)** processing block: frame length 0.025 s, frame stride 0.010 s, 40 filter bands, 512-point FFT.

• Add a **Classification (Keras)** learning block with 8 output classes.

5. **Model Architecture Specification:**

• In the Keras learning block, switch to **Expert mode** and paste the exact model architectures defined in `src/models.py`.

• Set training parameters: 10 epochs, learning rate 0.001, batch size 64.

6. **Quantization and Deployment:**

• After training completes, inspect the **Confusion matrix** and **Feature explorer**.

• Navigate to the **Deployment** tab. Select **C++ library**.

• Select the **Quantized (int8)** optimization option.

• Click **Build** to download the complete compiled C++ inference engine including model weights, CMSIS-NN kernels, and feature extraction pipelines.

---

11. Conclusion

The Edge Health Monitor successfully validates the thesis that complex, multi-modal healthcare monitoring systems can execute within the rigorous computational, memory, and energy envelopes of commercial microcontrollers. By synthesizing compact neural architectures (1D-CNN and DS-CNN), post-training integer quantization (INT8 PTQ), heuristic physical fallback rules, and debounced state management, the system achieves dependable detection of dangerous clinical events (falls and verbal distress) while preserving patient privacy and minimizing wireless communication overhead. Future work will center on physical hardware deployment and clinical evaluation with wearable inertial sensor hardware in geriatric rehabilitation facilities.